

Feynman Technical Guide — MotaDSLUM

Table of Contents

The Problem

The Scale of the Boilerplate

The Recipe Analogy

What a Solution Looks Like

The Question This Answers

The Big Picture

The Postal Service Analogy

System Boundaries

The Two Flows

- Generation Flow

- Runtime Flow

Component Responsibilities

The Subject Hierarchy

The NATS Micro Bridge

What the System Does Not Do

The DSL

Why Not Just Write Go?

- Three Pillars of the DSL Approach

MPS for Go Developers

- The Entity Concept

- The Relation Concept

- The Hook Concept

- TextGen: The Recipe Book

From DSL Declaration to Running Code

The Pipeline

The Walled Garden

- The Wrong Escape Route: Find a Better Plugin

- The Right Solution: `parse_textgen.py`

The One-Concept-One-File Constraint

- The Wrong Model: Single-File Everything

- The Right Model: Separate Root Concepts

Stage 1: MPS TextGen Output

Stage 2: `merge_hooks.py` — Hook Extraction and Merge

- War Story: The Signature Mismatch

Stage 3: `sync.sh` — Copy, Merge, Patch, Build

- The SDK Module Path Problem

Stage 4: Docker Compose

The Full Pipeline in One View

The Developer Flow

Step 1: Define the Entity in MPS

Step 2: Define Hooks

Step 3: Define Relations

Step 4: Run TextGen

Step 5: Run sync.sh

War Story: The Trailing Dot Bug

Step 6: Run demo.sh

What Can Go Wrong at Each Step

The User Flow

The Subject Hierarchy

Tracing One Request End-to-End

Hook Execution Model

Service Discovery

Error Patterns

Lessons and Red Flags

Lesson 1: The Single-File Trap

Incident

Forensics

Principle

Signal

Lesson 2: The Hook Name Collision

Incident

Forensics

Principle

Signal

Lesson 3: Presence Is Not Equivalence

Incident

Forensics

Principle

Signal

Lesson 4: Silent Generation Failures

Incident

Forensics

Principle

Signal

Lesson 5: Don't Fight the Walled Garden — Build a Bridge

Incident

Forensics

Principle

Signal

Lesson 6: External Scripts Are First-Class Tooling

Incident

Forensics

Principle

Signal

Principles Reference Card

The Problem

The UserManagement service manages three entities: User, Roles, Permissions. Each entity supports five operations: create, get, update, delete, list. That is fifteen handler methods.

Each handler method follows the same structure:

1. Start an OTEL span.
2. Unmarshal the incoming NATS message body into a typed event struct.
3. Run all pre-hooks for this operation, in priority order, distinguishing sync (can abort) from async (fire-and-forget).
4. Validate required fields.
5. Set span attributes.
6. Forward the request to the DAL via NATS request-reply with a 30-second timeout.
7. Run all post-hooks, distinguishing sync (transforms the response) from async (fire-and-forget).
8. Reply to the original NATS message.

That is eight steps, repeated fifteen times across three files. On top of that, two relations (UserRoles, RolesPermissions) add six more handler methods with a similar but distinct structure.

Twenty-one handler methods total. The structure is identical across all of them except for the type names and subject strings.

The Scale of the Boilerplate

```
forServer: motadata_user_management
entity: User
```

fields:

```
- ID          type: uuid      anotations: @ primaryKey
- password    type: password  anotations: @ required
                                     @ hidden
- UserName    type: string    anotations: @ required
- userMail    type: email     anotations: @ unique
- createdAt   type: time      anotations: @ auto
```

allowed_to:

```
- create
- update
- delete
- get
- list
```

preHooks:

create

```
- name: D - priority: 4 isAsync: false
- name: C - priority: 3 isAsync: true
- name: B - priority: 2 isAsync: false
- name: M - priority: 9 isAsync: true
- name: A - priority: 1 isAsync: true
```

delete

```
- name: Xyz          - priority: 5 isAsync: false
- name: Abc           - priority: 4 isAsync: false
- name: validateTen  - priority: 3 isAsync: false
```

update

```
- name: notifyCache - priority: 2 isAsync: true
- name: auditLog    - priority: 1 isAsync: true
```

get

```
- name: traceAcces - priority: 1 isAsync: true
```

postHooks:

```
..
```

agmentMps > UserManagement.sandbox > sandbox > User > ⓘ Field

The UserManagement service requires, per entity:

Artifact	Lines	Content
Handler code	~595	CRUD operations, hook dispatch, validation
SQL schema	8	Table definition with constraints
Hook stubs	30+	4 signatures × multiple operations
Integration tests	21	End-to-end pipeline validation

For three entities (User, Roles, Permissions): **1,790 lines of Go.**

A developer adding a fourth entity — say, **Tenants** — must write another 400+ lines of Go following this recipe exactly. Miss step 3 and pre-hooks silently do not run. Miss step 7 and post-hooks silently do not run. Get the DAL subject string wrong and the service appears to work until a specific operation triggers a silent routing failure.

The structure is a recipe. The problem is that the recipe is held in a developer's head rather than written down in a form a machine can execute.

The Recipe Analogy

A professional kitchen does not rely on every chef memorizing the same recipes. It writes them down. Consistency comes from the recipe, not from the chef's memory.

The equivalent in software is a template. When every entity handler follows the exact same structure — unmarshal, pre-hooks, validate, DAL forward, post-hooks, respond — that structure is a recipe. The specific entity (User, Roles, Permissions) is an ingredient list.

Hand-writing 1,790 lines of Go from a mental recipe carries specific failure modes:

- Mistakes that a text editor cannot catch (wrong subject string, missing hook call, copy-paste with the wrong type name).
- Every new entity means another ~400 lines of hand-written boilerplate.
- When the recipe changes — OTEL span attributes need to appear in a new place — every handler must be updated by hand.

The recipe should be written down once, in a form a computer can execute.

What a Solution Looks Like

The right solution separates *what an entity is* from *how a handler works*.

A developer declares:

```
Entity: User
  fields: id (uuid, primaryKey), username (string), email (email), password (password, hidden)
  operations: create, get, update, delete, list
  hooks on create: preCreated (pre-sync), preCreateM (pre-async), postCreateNotifyAdmin (post-async)
```

A code generator reads this declaration and produces the 400-line handler file, correctly, every time. The developer never touches the generated file. When the recipe changes, the generator changes — and all handlers regenerate correctly.

This is what the UserManagement DSL does. The MPS projectional editor holds the declaration. TextGen templates encode the recipe. `sync.sh` runs the pipeline. The result is 1,790 lines of correct, consistent Go code that no developer hand-writes or hand-maintains.



Design rationale: The DSL does not generate *all* Go code. It generates the structural scaffolding — the handler skeletons, the event types, the NATS wiring. Business logic lives in `userDefinedHooks.go`, which is preserved across regeneration by `merge_hooks.py`. The split is intentional: generated code should be disposable; business logic should not be.

The Question This Answers

What if adding a new entity required 15 lines of DSL instead of 300 lines of Go?

The answer is yes — with the UserManagement DSL, adding a fourth entity is approximately 15 lines of DSL declaration. The generator produces the handler file. No hand-writing. No missed steps.

The rest of this guide explains how the system achieves this, what constraints shape its architecture, and where the failure modes live.



Watch for:

- The urge to "just hand-write it this one time." It is always *this one time* until it is the fifteenth time.
- Generated code that is also manually edited. When the generator reruns, edits disappear. If you find yourself editing generated files, the right fix is to change the template or add a hook.
- Boilerplate growth that scales with entity count. If adding a new entity requires changes in N places proportional to the entity's operation count, the recipe exists but has no recipe book.

The Big Picture

The UserManagement service is a business logic layer. It sits between callers and a Data Access Layer (DAL), adding validation, hooks, and OTEL tracing to every operation. It does not store data — it processes requests and forwards them.

Understanding this positioning is the key to understanding every architectural decision that follows.

The Postal Service Analogy

Think of the system as a postal network.

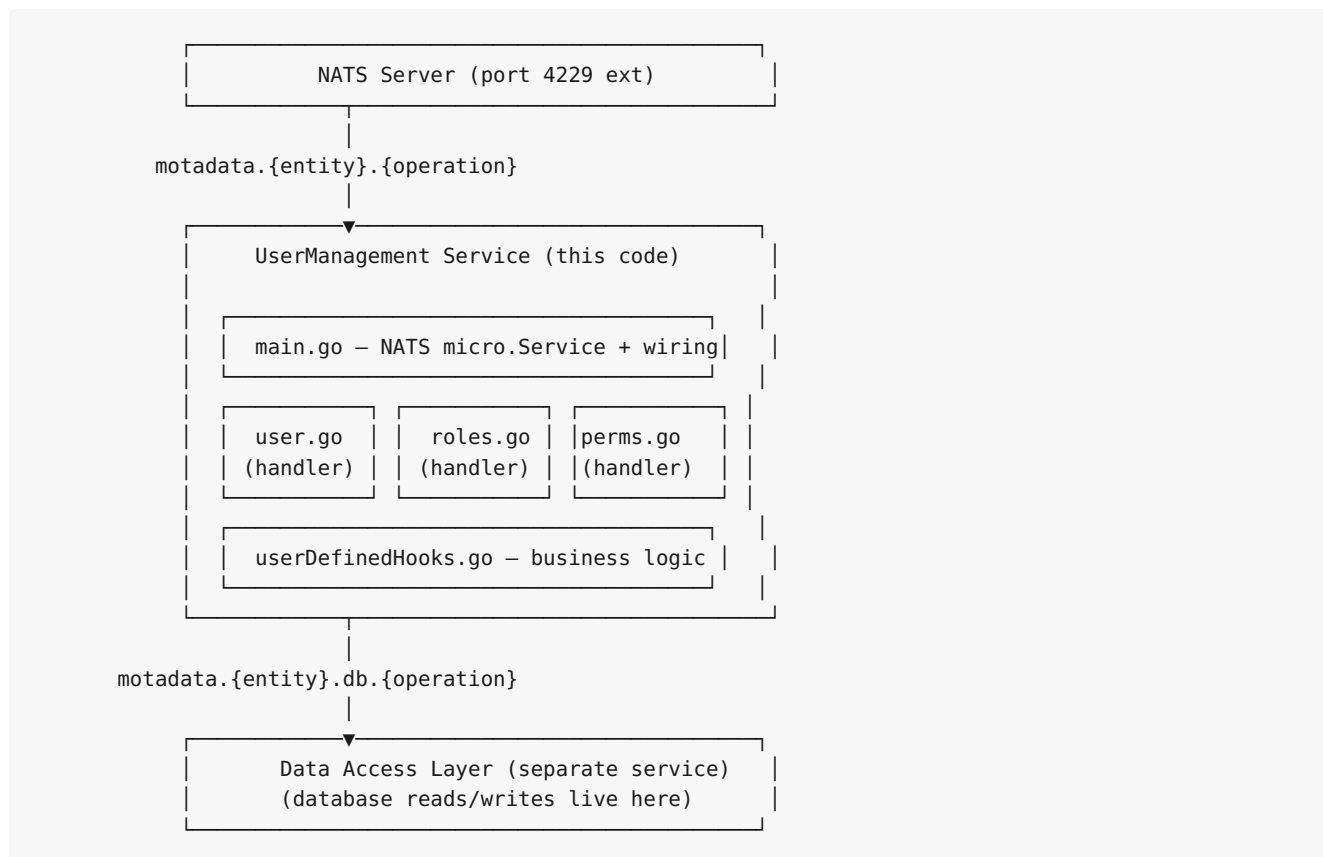
NATS subjects are street addresses. `motadata.user.create` is as specific as "123 Main Street." Any message sent to that address lands in exactly one mailbox — the User entity handler's `HandleCreate` method.

The handler is the mailbox. It receives the letter (incoming NATS message), checks it for completeness (validation), stamps it with business context (OTEL span attributes, hook results), and re-routes it to the warehouse address (the DAL subject `motadata.user.db.create`).

The DAL is the warehouse. It stores and retrieves the actual goods. The UserManagement service is the post office — it does not store anything, it processes and routes.

In this analogy, `demo.sh`'s mock DAL responders are a warehouse simulator: they accept any delivery and reply "received" without actually storing anything. This lets the entire postal network — addressing, routing, validation, hooks — be tested without a real warehouse.

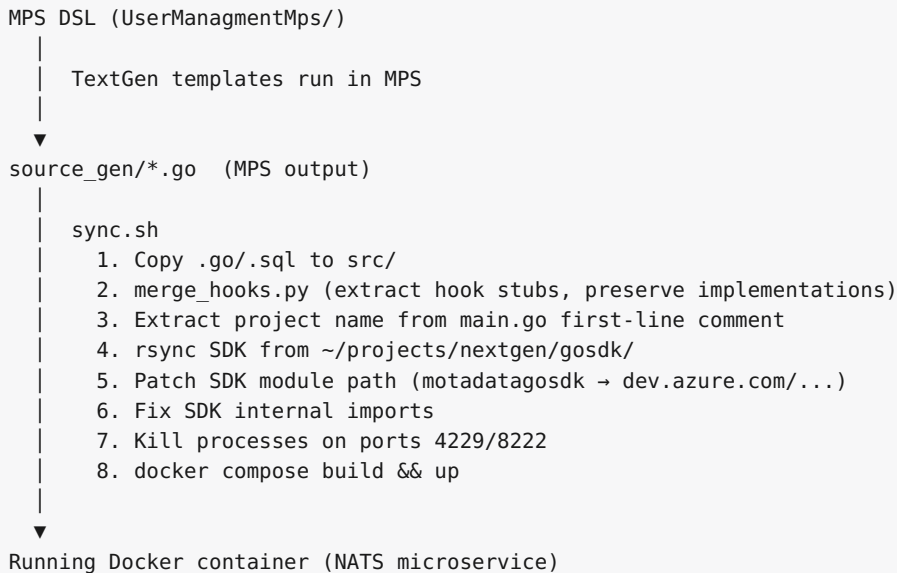
System Boundaries



The Two Flows

There are two distinct flows to understand: the *generation flow* (how the code comes to exist) and the *runtime flow* (what happens when a message arrives).

Generation Flow



The DSL is the single source of truth. Every file in `src/` except `userDefinedHooks.go` is generated — it will be overwritten the next time MPS regenerates and `sync.sh` runs. The only file safe to edit by hand is `userDefinedHooks.go`.

Runtime Flow

1. Client publishes: `nats req motadata.user.create '{"username":"alice","email":"alice@co.com",...}'`
2. `main.go` routes to User handler's `HandleCreate` method
3. `HandleCreate`:
 - a. Start OTEL span "user.create"
 - b. Unmarshal body → `UserCreatedEvent`
 - c. Pre-hooks (sorted by priority):
 - `preCreatedD (sync)` → validate or abort
 - `preCreateM (async)` → fire notification, continue
 - d. Validate required fields (returns error if missing)
 - e. Set span attributes (`user.username`, `user.email`, ...)
 - f. NATS request → `motadata.user.db.create` (30s timeout)
 - g. Post-hooks:
 - `postCreateNotifyAdmin (async)` → fire admin alert
 - h. Reply to original request with DAL response
4. DAL processes `motadata.user.db.create` and replies
5. Client receives response



Design rationale: The subject split — `motadata.user.create` for incoming, `motadata.user.db.create` for DAL — is not accidental. It means the UserManagement service can be inserted between any caller and any DAL without callers or the DAL needing to change their subject patterns. The business layer is a middleware that speaks both languages.

Component Responsibilities

Component	Responsibility
MPS DSL (UserManagmentMps/)	Single source of truth. Defines entities, fields, operations, relations, hooks, and the NatsServer configuration.
TextGen templates	Encode the handler recipe. Produce <code>user.go</code> , <code>roles.go</code> , <code>permissions.go</code> , <code>main.go</code> , and <code>sqlPrem_init_sql.sql</code> from DSL declarations.
<code>merge_hooks.py</code>	Extracts hook stubs from generated entity files, merges them into <code>userDefinedHooks.go</code> , and detects signature changes. Preserves user implementations across regeneration.
<code>sync.sh</code>	Orchestrates the full pipeline: copy from MPS, run merge, vendor+patch SDK, kill old containers, rebuild and restart.
<code>userDefinedHooks.go</code>	The only file safe to edit. Contains all business logic as hook implementations. Preserved across regeneration.
Docker Compose	Runs NATS (port 4229 external, 4222 internal) + the app container. The app connects to NATS at <code>nats://nats:4222</code> on the internal network.
<code>demo.sh</code>	21 end-to-end tests. Spawns mock DAL responders, sends <code>nats req</code> messages, verifies responses.

The Subject Hierarchy

Every NATS subject in the system follows two patterns:

Incoming (business layer): `motadata.{entity}.{operation}`

Examples: `motadata.user.create`, `motadata.roles.list`, `motadata.user.roles.assign`

Outgoing (DAL): `motadata.{entity}.db.{operation}`

Examples: `motadata.user.db.create`, `motadata.roles.db.list`

For relations, the pattern extends: `motadata.{from}.{to}.db.{operation}`

This hierarchy is not configured anywhere — it is *generated* by TextGen from the entity and relation names in the DSL. Change an entity name in MPS and the subject strings change everywhere automatically.

The NATS Micro Bridge

`main.go` uses the `nats.go/micro` package to expose the service over NATS's microservice protocol. This gives the service:

- Automatic service discovery via `$SRV.INFO`
- Structured endpoint registration per operation
- A standard way to inspect what subjects the service handles

The service name comes from the first-line comment in the generated `main.go` :

```
// MotaDSLUM
package main
```

GO

`sync.sh` extracts this comment with `head -1 src/main.go | sed 's|^// *||'` and passes it as a Docker build argument. Single source of truth: the DSL's `NatsServer` concept generates the comment, and everything downstream (Dockerfile, container name) derives from it.



Design rationale: Using a first-line comment as the service name source is pragmatic, not elegant. A config file would be cleaner, but the comment is *generated automatically* from the DSL's `NatsServer` concept — no manual configuration required. The generator writes it; the pipeline reads it.

What the System Does Not Do

Understanding system boundaries prevents surprises:

- **No persistence.** The UserManagement service does not connect to a database. All reads and writes go through the DAL.
- **No authentication.** Auth is handled upstream. The service assumes messages arrive from trusted callers.
- **No retry logic.** NATS request-reply uses a 30-second timeout. If the DAL does not respond, the call fails. No retry.
- **No schema migrations.** The generated `sqlPrem_init_sql.sql` creates the initial schema. Migrations are out of scope.



Watch for:

- Editing files in `src/` that are not `userDefinedHooks.go`. They will be overwritten on the next `sync.sh` run.
- Hardcoded subject strings in hook implementations. If the entity name changes, generated subjects change but hardcoded strings in hooks do not.
- Missing mock DAL responders in test scripts. If `demo.sh` does not spawn a responder for a subject, tests for that entity will time out rather than fail with a clear error.

The DSL

Why Not Just Write Go?

```
forServer: motadata_user_management
entity: User
```

fields:

```
- ID          type: uuid      anotations: @ primaryKey
- password    type: password anotations: @ required
               @ hidden
- UserName    type: string    anotations: @ required
- userMail    type: email     anotations: @ unique
- createdAt   type: time      anotations: @ auto
```

allowed_to:

```
- create
- update
- delete
- get
- list
```

preHooks:

create

```
- name: D - priority: 4 isAsync: false
- name: C - priority: 3 isAsync: true
- name: B - priority: 2 isAsync: false
- name: M - priority: 9 isAsync: true
- name: A - priority: 1 isAsync: true
```

delete

```
- name: Xyz          - priority: 5 isAsync: false
- name: Abc          - priority: 4 isAsync: false
- name: validateTen - priority: 3 isAsync: false
```

update

```
- name: notifyCache - priority: 2 isAsync: true
- name: auditLog    - priority: 1 isAsync: true
```

get

```
- name: traceAcces - priority: 1 isAsync: true
```

postHooks:

```
..
```

agmentMps > UserManagement.sandbox > sandbox > User > (N) Field

The structural problem with hand-writing entity handlers is not the typing — it is the mental model that must be held simultaneously: "in `HandleCreate`, step 3 is pre-hooks sorted by priority, step 4 is validation, step 5 is span attributes."

Miss step 3 and pre-hooks silently do not fire. Get the order wrong and the validator runs before hooks can abort, which means aborted requests still reach the DAL.

The failure mode has a name: *maintaining consistency through discipline*. It fails as the codebase grows because humans make mistakes under repetition.

The right approach is to encode the recipe once, in a form a machine can execute reliably. That form is a DSL.

Three Pillars of the DSL Approach

Consistency. Every handler follows the exact same eight-step structure because they are all produced by the same template. It is not possible to accidentally skip step 3 in `roles.go` but not `user.go` — the template either generates it everywhere or nowhere.

Single source of truth. The entity name `User` appears in the Go struct, the event types (`UserCreatedEvent`, `UserGetRequest`), the NATS subjects (`motadata.user.create`), the SQL table name (`iam.user`), and the span attributes (`user.username`). In hand-written code, these are four separate places to update. In the DSL, they derive from one declaration.

Generation velocity. Adding a fourth entity (say, `Tenants`) requires writing ~15 lines of DSL. The generator produces the handler file. No hand-writing. No missed steps.



Trade-offs honestly stated: The DSL approach costs something real. Learning MPS is a non-trivial investment with a steep initial curve. The TextGen templates must be maintained — a bug in the template affects all generated files simultaneously. And debugging generated code requires tracing backward from the output to the template node that produced it, without line numbers or breakpoints.

These are real costs. The system is worth it when the entity count is growing and consistency matters more than flexibility. It is not worth it for a one-off handler.

MPS for Go Developers

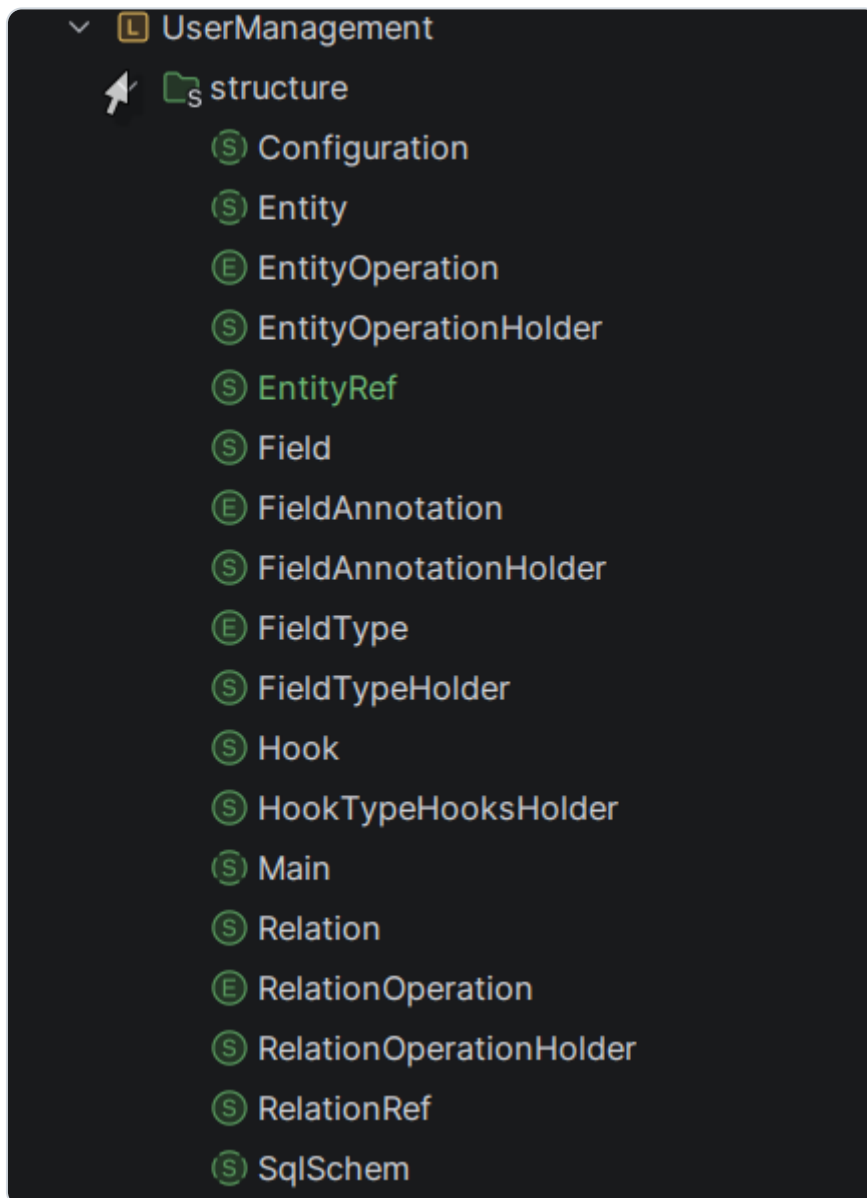
MPS is a projectional editor, not a text editor. This is the most important thing to understand before anything else.

In a text editor, you write characters and the parser turns them into a syntax tree. In MPS, you directly edit the syntax tree — the tree is the document. There are no characters in between.

The analogy: a spreadsheet versus a word processor. In a word processor, you type `"=SUM(A1:A10)"` and the parser interprets it. In a spreadsheet, you click a cell, click a function button, and the spreadsheet builds the formula structure for you. MPS is the spreadsheet.

This has a profound consequence: there is no copy-paste from an external text editor into MPS's TextGen buffers. Building a template character by character through MPS's completion menus is the default path — and the walled garden problem this creates is addressed in [the-pipeline].

The Entity Concept



An Entity in the DSL represents a domain object — not a database table, not a Go struct, but a business concept with operations and behavior attached.

```
Entity: User
fields:
  id          uuid          primaryKey, auto
  username    string        required
  email       email         required      (implies UNIQUE in SQL)
  password    password      hidden          (excluded from JSON responses)
  created_at  time           auto
  updated_at  time           auto
operations: create, get, update, delete, list
hooks on create:
  preCreateD (pre-sync, priority 1)
  preCreateM (pre-async, priority 2)
  postCreateNotifyAdmin (post-async, priority 1)
```

From this declaration, TextGen produces:

```
// User struct (from field declarations)
type User struct {
    Id          string `json:"id" db:"id"`
    UserName    string `json:"user_name" db:"user_name"`
    UserMail    string `json:"user_mail" db:"user_mail"`
    // Password has no json tag – the "hidden" annotation removes it
    Password    string `db:"password"`
    CreatedAt   string `json:"created_at" db:"created_at"`
    UpdatedAt   string `json:"updated_at" db:"updated_at"`
}

// Event types (one per operation)
type UserCreatedEvent struct { ... }
type UserGetRequest struct { ... }
type UserUpdatedEvent struct { ... }
type UserDeletedEvent struct { ... }
type UserListRequest struct { ... }

// Handler struct
type UserHandler struct {
    natsConn core.Connection
    publisher core.Publisher
    tracer    tracer.Tracer
    logger    logger.Logger
    // workerpool included only if any hook is async
    wp        *workerpool.WorkerPool
}

// Five Handle methods follow...
```

The DSL's field types carry domain semantics that plain Go types do not:

DSL Type	Go/SQL equivalent	What it implies
uuid	string / UUID	Auto-generated via <code>gen_random_uuid()</code> , primary key candidate
email	string / VARCHAR(255)	UNIQUE constraint in SQL, validation hook target
password	string	hidden annotation: excluded from JSON, policy checking target
time	string / TIMESTAMPTZ	auto means <code>DEFAULT NOW()</code> in SQL
json	string / JSONB	PostgreSQL JSONB storage
entityRef	string / UUID REFERENCES	Foreign key; DAL handles the join

This represents the evolution from SQL-primitive types (`int64`, `string`, `float`) to domain types. With primitive types, `email` is just `string` — the developer is responsible for adding the UNIQUE constraint and writing validation separately. With domain types, `email` is the constraint.

The Relation Concept

```
forServer: motadata_user_management
entity: Roles

fields:
- ID      type: uuid   anotations: @ primaryKey
- desc    type: text   anotations: @ nullable
- name    type: string anotations: @ required

allowed_to:
- create
- update
- list
- delete
- get

preHooks:
  create
    - name: NotifyAdmin - priority: 10 isAsync: true
postHooks:
  << ... >>

relations:
relation Permission_pivot {
  with: Permissions

  operations:
    -assign
    -list
    -remove

  extra fields:
    << ... >>
}
```

A Relation connects two entities with a named set of operations: assign, remove, and list.

```
Relation: UserRoles
from: User
to: Roles
operations: assign, remove, list
extraFields:
  assigned_at time auto
```

From this, TextGen produces:

- Three event types: `UserRolesAssignedEvent`, `UserRolesRemovedEvent`, `UserRolesListRequest`

- Three handler methods inside `user.go` : `HandleAssign`, `HandleRemove`, `HandleList` (for the relation)
- A junction table in SQL: `iam.user_roles` with composite primary key and cascade deletes

Relations are generated into the *same file* as their owning entity (`UserRoles` → inside `user.go` , `RolesPermissions` → inside `roles.go`). This is a consequence of MPS's one-concept-one-file constraint. See [the-pipeline] for the full story.

The Hook Concept

```
preHooks:
  create
    - name: D - priority: 4 isAsync: false
    - name: C - priority: 3 isAsync: true
    - name: B - priority: 2 isAsync: false
    - name: M - priority: 9 isAsync: true
    - name: A - priority: 1 isAsync: true
  delete
    - name: Xyz - priority: 5 isAsync: false
    - name: Abc - priority: 4 isAsync: false
    - name: validateTen - priority: 3 isAsync: false
  update
    - name: notifyCache - priority: 2 isAsync: true
    - name: auditLog - priority: 1 isAsync: true
  get
    - name: traceAcces - priority: 1 isAsync: true
postHooks:
  update
    - name: invalidateCache - priority: 1 isAsync: true
  get
    - name: filterPII - priority: 3 isAsync: false
    - name: cacheResult - priority: 2 isAsync: true
    - name: enrichData - priority: 1 isAsync: false
  create
    - name: notifyAdmin - priority: 2 isAsync: true
    - name: sendWelcome - priority: 1 isAsync: true
  delete
    - name: mustDelete - priority: 2 isAsync: false

relations:
relation UserRoll {
  with: Roles

  operations:
    -assign
    -list
    -remove

  extra fields:
    - assignedAt type: time anotations: @ auto
}
```

A Hook is a named extension point attached to a specific operation. The 2×2 matrix of hook variants:

	Pre-operation	Post-operation
Sync (blocks, can abort)	func (s *Handler) hookName(ctx, span, event) error	func (s *Handler) hookName(ctx, span, event, data []byte) []byte
Async (fire-and-forget)	func (s *Handler) hookName(ctx, span, event)	func (s *Handler) hookName(ctx, span, event, data []byte)

The four signatures exist because there are four distinct use cases:

- **Pre-sync:** Validation and authorization. Returns an error to abort the operation before it reaches the DAL.
- **Pre-async:** Audit logging, notifications. Fires without blocking — the operation continues whether or not the async work completes.
- **Post-sync:** Data transformation and PII filtering. Receives the DAL response as `[]byte`, returns the modified response.
- **Post-async:** Cache invalidation, admin alerts. Fires after the DAL response without blocking the reply to the caller.



Design rationale: The four signatures emerged from customer extensibility requirements. Customers need async audit trails (pre-async) *and* sync validation (pre-sync) on the same operation. A single `hook(ctx, event) ([]byte, error)` signature cannot express both — you cannot make a function both blocking-returnable and fire-and-forget simultaneously. The 2×2 separation is intentional: each axis (pre/post, sync/async) is an independent orthogonal choice.

Hook priority controls execution order within a phase. Lower priority number runs first within a phase:

```
hooks on get:
  filterPII      post-sync  priority 3
  cacheResult    post-sync  priority 2
  enrichData     post-sync  priority 1
```

Hooks execute in order: `enrichData` (1) → `cacheResult` (2) → `filterPII` (3).



Critical ordering rule: Always place sync validator hooks at a *lower priority number* (runs first) than async hooks on the same operation. Async hooks are fire-and-forget — if a sync validator fires *after* an async hook, the async hook has already started even if the sync validator will eventually abort the operation. The async hook keeps running. It cannot be cancelled.

TextGen: The Recipe Book

TextGen is the MPS mechanism for describing code generation templates. Think of it as a smart template engine with type checking.

A TextGen component declares: "for every instance of concept Entity in the sandbox, generate one `.go` file at path `/src/{entity.name.toLowerCase()}.go` containing the following content."

The "following content" is expressed as a sequence of `append` statements:

```
text gen component for concept Entity {
  file path: "/src/"
  extension: "go"
  file name: node.name.toLowerCase()

  body:
    append {package main\n} ;
    append {\nimport (\n} ;
    append {      "context"\n} ;
    // ... conditional imports based on entity properties ...
    foreach operation in node.operations {
      append {\nfunc (s *} ;
      append ${node.name} ;
      append {Handler) Handle} ;
      append ${operation.name.capitalize()} ;
      append {(msg micro.Request) {\n} ;
      // ... eight-step handler body ...
    }
  }
}
```

Each `append {...}` statement was typed character-by-character in MPS's projectional editor (before `parse_textgen.py` existed). The Entity TextGen template is approximately 300 `append` statements producing ~400 lines of Go per entity.

The template supports conditional logic:

```
// Only import workerpool if any hook is async
append ${node.needWorkerPool() ? "\"...workerpool\"" : ""} \n ;

// Only validate non-auto, non-hidden, non-nullable fields
foreach field in node.fields {
  if (!(field.hasAnnotation("primaryKey")) && !(field.hasAnnotation("auto"))
      && !(field.hasAnnotation("hidden")) && !(field.hasAnnotation("nullable"))) {
    append {|| event.} ;
    append ${field.name.capitalize()} ;
    append { == ""} ;
  }
}
```

This is why `password` fields are excluded from validation (they are `hidden`) and `id` fields are excluded (they are `primaryKey` and `auto`). The template knows which fields to skip because the DSL field annotations carry that information.

From DSL Declaration to Running Code

The full chain for a single entity:

1. Developer defines `User` in MPS with fields, operations, and hooks.
2. TextGen runs, producing `user.go` (595 lines) and `sqlPrem_init_sql.sql` (partial).
3. `sync.sh` copies `user.go` to `src/`.
4. `merge_hooks.py` extracts hook stubs from `user.go` into `userDefinedHooks.go`.
5. Developer implements business logic in `userDefinedHooks.go`.
6. `sync.sh` rebuilds the Docker container.

7. `demo.sh` verifies the handler works end-to-end.

Adding a field to `User` in MPS restarts at step 2. `userDefinedHooks.go` is preserved at step 4 — the developer's implementations are not overwritten.

Watch for:



- Typos in DSL field names propagate faithfully to generated Go struct fields and NATS subject strings. `motadata.usr.create` (missing 'e') will be silently generated if the entity is named `Usr`.
- DSL constraints are not automatically inferred. If a field should be unique, it must be declared as `email` type (which implies unique) or given an explicit unique annotation. The generator trusts the declaration.
- The Holder Concept Pattern — an artifact of incomplete MPS editor styling knowledge — adds unnecessary nesting in some entity definitions. If you see a `HooksHolder` wrapper concept between `Entity` and `Hook`, it is a workaround, not a feature. Direct `0..n` children with visibility queries would be cleaner.

The Pipeline

The pipeline is the machinery between a DSL declaration and a running Docker container. It is not one script — it is a chain of tools, each solving a problem the previous tool could not.

Understanding the chain requires understanding *why* each tool exists, which means understanding the constraints that made the naive approach fail.

The Walled Garden

The central constraint of the entire pipeline is MPS's projectional editor.

In a text editor, templates are text files. They can be written in any editor, AI-assisted, copy-pasted from working code, and versioned as plain text in git.

In MPS, templates are stored as binary `.mps` XML files. They are authored through MPS's completion menus — not by typing text, but by selecting nodes and invoking editor actions. There is no way to paste a Go handler from an external editor into a TextGen buffer. There is no way to copy TextGen content out to a plain text file. There is no way to use an AI assistant to help write the template.

This is the walled garden. Everything inside MPS is inaccessible to external tooling. Everything external to MPS cannot enter.

Three walls:

1. **No import:** Cannot paste code snippets into TextGen buffers. Every character must be typed through MPS completion menus.
2. **No export:** Cannot copy TextGen content to an external file for editing or AI assistance.
3. **No tooling:** Cannot use scripts, AI, or any external tool to modify templates directly.

For a 300- append -statement template producing 400 lines of Go per entity, the walled garden means weeks of manual authoring.

The Wrong Escape Route: Find a Better Plugin

The first attempt to escape the walled garden was to switch from native MPS TextGen to the DSL Foundry TextGen plugin.

The attempt failed. Documentation was sparse. The groupings model — how DSL Foundry structures template output — was opaque without guides. After significant time invested, the attempt was abandoned without producing a working template.

The lesson: switching plugins is not always the right escape route. Sometimes the right answer is to build a bridge.

The Right Solution: parse_textgen.py

`parse_textgen.py` is a Python script that reverse-engineers the MPS TextGen XML format.

The approach:

1. Examine existing TextGen `.mps` files from templates that already work.
2. Map each TextGen node type to its XML representation.
3. Build a parser that accepts human-readable markdown files that *look like* TextGen templates.
4. Convert the markdown to valid `.mps` XML.
5. Load the `.mps` file into MPS as if it were authored natively.

TEXT

```
# What you write (plain markdown):
package ${node.name.toLowerCase()}\n

import (\n
    ${node.needWorkerPool() ? '"workerpool"' : ''}\n
)\n

# What parse_textgen.py produces (MPS XML, abbreviated):
<node concept="AppendOperation">
  <property role="text" value="package " />
  <node concept="NodeReference">
    <ref role="node" to="..." />
    <node concept="GetProperty">
      <property role="method" value="toLowerCase" />
    </node>
  </node>
  <property role="text" value="\n" />
</node>
```

The reverse engineering consumed one full day including bug fixing. The hardest part: each TextGen node type has a different child arrangement in the XML. String interpolations (`${...}`) have a different structure than literal text. Conditional expressions have a different structure than loops. Each combination required its own mapping rule.

The payoff: writing a new TextGen template becomes a matter of authoring a plain markdown file. AI assistance, copy-paste from working Go code, external editors — all become available again. TextGen changes are also diffable in git: markdown diffs instead of binary `.mps` diffs.



Design rationale: `parse_textgen.py` is a bridge, not a replacement. MPS still runs TextGen — it reads the `.mps` file and executes generation as normal. The bridge only changes how templates are *authored*.

The One-Concept-One-File Constraint

```
forServer: motadata_user_management
entity: User
```

fields:

```
- ID          type: uuid      anotations: @ primaryKey
- password    type: password  anotations: @ required
                                     @ hidden
- UserName    type: string    anotations: @ required
- userMail    type: email     anotations: @ unique
- createdAt   type: time      anotations: @ auto
```

allowed_to:

```
- create
- update
- delete
- get
- list
```

preHooks:

create

```
- name: D - priority: 4 isAsync: false
- name: C - priority: 3 isAsync: true
- name: B - priority: 2 isAsync: false
- name: M - priority: 9 isAsync: true
- name: A - priority: 1 isAsync: true
```

delete

```
- name: Xyz          - priority: 5 isAsync: false
- name: Abc          - priority: 4 isAsync: false
- name: validateTen - priority: 3 isAsync: false
```

update

```
- name: notifyCache - priority: 2 isAsync: true
- name: auditLog    - priority: 1 isAsync: true
```

get

```
- name: traceAcces - priority: 1 isAsync: true
```

postHooks:

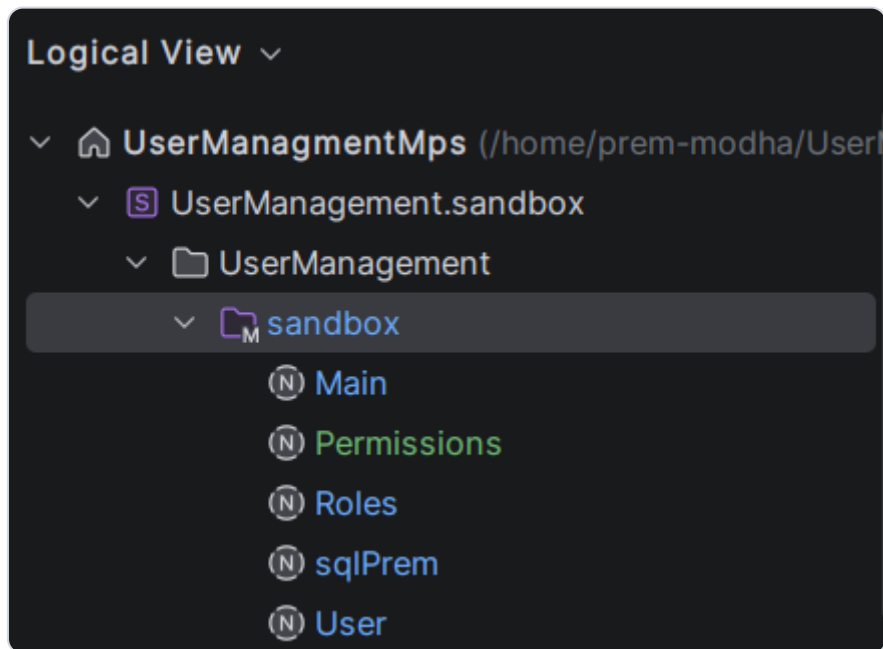
```
..
```

agmentMps > UserManagement.sandbox > sandbox > User > ⓘ Field

MPS imposes: one root concept instance in the sandbox produces one output file from TextGen.

This constraint reveals itself when separate files are required.

The Wrong Model: Single-File Everything



The natural instinct — especially coming from Prisma — is to define all entities in one place:

```
Models {  
  Entity: User { ... }  
  Entity: Roles { ... }  
  Entity: Permissions { ... }  
}
```

With MPS TextGen, this produces *one file* containing all three entity handlers combined. UMAN made this mistake — its TextGen file grew to 560 KB, the largest of all projects. `pcrd` made the same mistake: it generated working end-to-end code (Go Gin server, SQL schema, Docker setup) but all entities landed in one giant file.

The Right Model: Separate Root Concepts

```
forServer: motadata_user_management
entity: Roles

fields:
- ID      type: uuid   anotations: @ primaryKey
- desc    type: text   anotations: @ nullable
- name    type: string anotations: @ required

allowed_to:
- create
- update
- list
- delete
- get

preHooks:
  create
    - name: NotifyAdmin - priority: 10 isAsync: true
postHooks:
  << ... >>

relations:
relation Permission_pivot {
  with: Permissions

  operations:
    -assign
    -list
    -remove

  extra fields:
    << ... >>
}
```

To get `user.go`, `roles.go`, and `permissions.go` as separate files, each entity must be a separate root concept in the MPS sandbox:

Sandbox:

Main	(root) → main.go
User	(root) → user.go (includes UserRoles relation handlers)
Roles	(root) → roles.go (includes RolesPermissions relation handlers)
Permissions	(root) → permissions.go
SqlSchem	(root) → sqlPrem_init_sql.sql

Each root concept = one TextGen execution = one output file.

Relations are children of their owning entity, not standalone root concepts. `UserRoles` lives inside `User` and generates into `user.go`. This is why relation handlers appear in the same file as entity handlers — not by coincidence, but by constraint.



Design rationale: The `Main` concept does not *contain* entities — it *references* them via `EntityRef` nodes. This keeps entity definitions self-contained while allowing `main.go` to know which handlers to wire up.

Stage 1: MPS TextGen Output

When MPS runs TextGen on the sandbox, it produces raw Go files with `#HOOKS_START` / `#HOOKS_END` markers embedded:

GO

```
// user.go (generated – DO NOT EDIT)

// ... UserHandler struct, constructor, Handle methods ...

// #HOOKS_START
func (s *UserHandler) preCreated(ctx context.Context, span tracer.Span,
    event *UserCreatedEvent) error {
    return nil
}

func (s *UserHandler) preCreateM(ctx context.Context, span tracer.Span,
    event *UserCreatedEvent) {
}

func (s *UserHandler) postCreateNotifyAdmin(ctx context.Context, span tracer.Span,
    event *UserCreatedEvent, data []byte) {
}
// #HOOKS_END
```

The markers exist because `userDefinedHooks.go` must be a separate file — but MPS cannot generate it directly without creating a cluttering root concept. The solution: generate stubs *inside* entity files, then extract them externally.

Stage 2: merge_hooks.py — Hook Extraction and Merge

`merge_hooks.py` reads every generated entity file, extracts the stubs between `#HOOKS_START` and `#HOOKS_END`, and merges them into `userDefinedHooks.go`.

The merge has three outcomes per hook stub:

Outcome	Condition
Add	Hook stub is new — not yet in <code>userDefinedHooks.go</code> . Appended.
Skip	Hook stub already exists in <code>userDefinedHooks.go</code> with matching signature. Developer’s implementation is preserved.
Replace	Hook stub exists but signature has changed (sync ↔ async flip). Stub is replaced; old implementation is printed as a warning.

Identity key: `*ReceiverType.funcName` (e.g., `*UserHandler.preCreated`). This prevents collisions when multiple entity handlers have hooks with the same name — `*UserHandler.preCreate` and `*RolesHandler.preCreate` are distinct keys.

War Story: The Signature Mismatch

A hook was changed from post-sync to post-async. The signature changed from:

```
// Before: post-sync – transforms response, returns []byte
func (s *UserHandler) postCreate(ctx context.Context, span tracer.Span,
    event *UserCreatedEvent, data []byte) []byte {
    // developer's implementation
    return modifiedData
}
```

GO

to:

```
// After: post-async – fire-and-forget, no return value
func (s *UserHandler) postCreate(ctx context.Context, span tracer.Span,
    event *UserCreatedEvent, data []byte) {
}
```

GO

The name stayed the same. Before signature change detection was added, `merge_hooks.py` kept the old implementation — with the wrong return type. The build broke with a type error.

The fix: compare signatures on every merge, not just names. When a mismatch is detected, the old implementation is printed to stdout as a warning and the stub is replaced. The developer sees the warning and migrates the logic manually.

Principle: Presence is not equivalence. A function with the right name and the wrong shape is not the right function.

Stage 3: `sync.sh` — Copy, Merge, Patch, Build

`sync.sh` orchestrates the full pipeline in eight steps:

```
# Step 1: Copy .go and .sql from MPS source_gen/ into src/
cp source_gen/*.go src/
cp source_gen/*.sql src/

# Step 2: Run merge_hooks.py
python3 merge_hooks.py

# Step 3: Extract project name from main.go first-line comment
APP_NAME=$(head -1 src/main.go | sed 's|^// *||')

# Step 4: Sync vendored SDK
rsync -av ~/projects/nextgen/gosdk/motadata-go-sdk/ motadata-go-sdk/

# Step 5: Patch SDK module path (motadatagosdk → dev.azure.com/...)
sed -i 's|^module motadatagosdk|module dev.azure.com/Motadata/NextGen/motadata-go-sdk|' \
    motadata-go-sdk/go.mod

# Step 6: Fix SDK internal imports
find motadata-go-sdk -name '*.go' -exec sed -i \
    's|"motadatagosdk|"dev.azure.com/Motadata/NextGen/motadata-go-sdk|g' {} +

# Step 7: Kill lingering processes on service ports
fuser -k 4229/tcp 8222/tcp 2>/dev/null || true

# Step 8: Build and start Docker Compose
docker compose build --build-arg APP_NAME="$APP_NAME"
docker compose up -d
```

The SDK Module Path Problem

The Motadata Go SDK is vendored — copied locally — because it is not published to a Go module registry.

The SDK source uses `motadatagosdk` as its module name. The generated Go files import `dev.azure.com/Motadata/NextGen/motadata-go-sdk/events/core`. These do not match — Go refuses to build.

`sync.sh` patches the mismatch in steps 5 and 6. The `sed` commands are idempotent: patching an already-patched file is safe, so every `sync.sh` run leaves the SDK in a consistent state.

When the SDK is eventually published to a registry, steps 4–6 can be removed entirely. The generated imports will resolve directly via `go mod`.



Design rationale: The project name flows from DSL → generated comment → `sync.sh` extraction → Docker build arg → container name. A config file would be cleaner, but the first-line comment is generated automatically from the DSL's `NatsServer` concept. Single source of truth: change the name in MPS and it propagates through the entire stack.

Stage 4: Docker Compose

Two services in `docker-compose.yml`:

- **nats:** `nats:2.10-alpine` with JetStream enabled, healthcheck on `:8222/healthz`
- **app:** Built from `Dockerfile`, waits for NATS to be healthy before starting

Port mapping: `* 4229:4222` — NATS client port (external 4229, internal 4222) `* 8222:8222` — NATS HTTP monitoring

The port mismatch (4229 vs 4222) exists because 4222 was already in use on the development machine. It is a local conflict resolution, not an architectural choice.

The app connects to `nats://nats:4222` on the internal Docker network. `demo.sh` connects to `nats://localhost:4229` from the host.

Step 7 in `sync.sh` (`fuser -k 4229/tcp 8222/tcp`) clears any lingering processes before each rebuild, preventing port conflicts from previous runs.

The Full Pipeline in One View

```
MPS DSL (UserManagmentMps/)
|
|   Author TextGen templates via markdown + parse_textgen.py
|   (escapes walled garden)
|
|   ▼
TextGen generates:
  source_gen/main.go           155 lines
  source_gen/user.go           595 lines + #HOOKS markers
  source_gen/roles.go          481 lines + #HOOKS markers
  source_gen/permissions.go    307 lines + #HOOKS markers
  source_gen/sqlPrem_init_sql.sql 31 lines
|
|   sync.sh step 1: copy to src/
|
|   ▼
src/*.go (copied, will be overwritten on next sync)
|
|   sync.sh step 2: merge_hooks.py
|   - extract stubs from #HOOKS markers
|   - add new / skip existing / replace changed-signature
|
|   ▼
src/userDefinedHooks.go 221 lines, 30+ stubs, preserved across regeneration
|
|   sync.sh steps 3-6: name extraction, SDK rsync + patch
|
|   ▼
motadata-go-sdk/ vendored, module path patched to dev.azure.com/...
|
|   sync.sh steps 7-8: kill old processes, docker compose build + up
|
|   ▼
Running container on port 4229 (NATS microservice)
|
|   demo.sh: 21 e2e tests with mock DAL
|
|   ▼
All 21 tests pass ✓
```



Watch for:

- Running `sync.sh` without MPS having regenerated first. The script copies from `source_gen/` — if those files are stale, the container is stale.
- Manually editing files in `src/` other than `userDefinedHooks.go`. Overwritten at step 1.
- Hook signature changes that go unnoticed. `merge_hooks.py` prints a warning but does not fail the build. Check stdout after every sync that modifies a hook definition in MPS.
- The `user_roless` double-s bug: junction table `iam.user_roless` has a doubled suffix because a hardcoded `"s"` string was appended to entity names that already ended in `s`. Always use DSL variables for name construction, not hardcoded string fragments.

The Developer Flow

```
forServer: motadata_user_management
entity: Permissions

fields:
- ID      type: uuid  anotations: @ required
                                     @ primaryKey
                                     @ immutable
- name    type: string anotations: @ required
- module  type: text   anotations: @ required
- can     type: string anotations: @ required

allowed_to:
- create
- update
- delete
- list
- get

preHooks:
  create
    - name: NotifyManager - priority: 100  isAsync: false
    - name: NotifyAdmin   - priority: 1000 isAsync: true
postHooks:
  create
    - name: audit - priority: 1  isAsync: true

relations:
<< ... >>
```

This section traces the complete workflow for adding a new entity to the system. The example is the Permissions entity — the third entity added after User and Roles were already working.

The six steps:

1. Define the entity in MPS (fields, operations)
2. Define hooks on operations
3. Define relations (RolesPermissions)
4. Run TextGen to generate Go files

5. Run `sync.sh` to copy, merge, patch, and rebuild

6. Run `demo.sh` to verify

Step 1: Define the Entity in MPS

```
forServer: motadata_user_management
entity: User
```

fields:

```
- ID          type: uuid      anotations: @ primaryKey
- password    type: password  anotations: @ required
                                     @ hidden
- UserName    type: string    anotations: @ required
- userMail    type: email     anotations: @ unique
- createdAt   type: time      anotations: @ auto
```

allowed_to:

```
- create
- update
- delete
- get
- list
```

preHooks:

create

```
- name: D - priority: 4 isAsync: false
- name: C - priority: 3 isAsync: true
- name: B - priority: 2 isAsync: false
- name: M - priority: 9 isAsync: true
- name: A - priority: 1 isAsync: true
```

delete

```
- name: Xyz          - priority: 5 isAsync: false
- name: Abc          - priority: 4 isAsync: false
- name: validateTen - priority: 3 isAsync: false
```

update

```
- name: notifyCache - priority: 2 isAsync: true
- name: auditLog    - priority: 1 isAsync: true
```

get

```
- name: traceAcces - priority: 1 isAsync: true
```

postHooks:

```
..
```

agmentMps > UserManagement.sandbox > sandbox > User > (N) Field

Open MPS, navigate to the UserManagementMps project, and create a new root node of type `Entity`.

The projectional editor presents a structured form — not a text file. Fill in the entity name, then add fields one by one using completion menus.

```
Entity: Permissions
fields:
  id          uuid      primaryKey, auto
  name        string    required
  description text      nullable
  created_at  time      auto
operations: create, get, update, delete, list
```

The DSL editor validates as you type. `uuid + primaryKey` tells the generator to produce `UUID PRIMARY KEY DEFAULT gen_random_uuid()` in SQL. `text + nullable` tells it to omit `NOT NULL`. `time + auto` produces `TIMESTAMP TZ NOT NULL DEFAULT NOW()`.

Compare this to the User entity:

```
Entity: User
fields:
  id          uuid      primaryKey, auto
  username    string    required
  email        email    required
  password    password  hidden
  created_at  time      auto
  updated_at  time      auto
operations: create, get, update, delete, list
```

The structure is identical — same field type vocabulary, same operation list, same annotation system. The generator sees the same pattern and applies the same template. The output will be a `permissions.go` file with the same eight-step structure as `user.go` and `roles.go`, with `Permissions`-specific type names and subjects.



The faithful propagation rule: Typos in DSL field names propagate without correction. If the entity is named `Permissons` (missing an 'i'), the generated file will be `permissons.go`, the NATS subject will be `motadata.permissons.create`, and the SQL table will be `iam.permissons`. The generator trusts the declaration. MPS constraints can catch some errors (duplicate names, missing required fields) but cannot catch semantic mistakes like misspellings.

Step 2: Define Hooks

With the entity defined, add hooks to specific operations. In MPS, hooks are children of an operation within the entity definition.

```
Entity: Permissions
...
hooks on create:
  preCreateValidate (pre-sync, priority 1) ← blocks; can abort
  preCreateAudit    (pre-async, priority 2) ← fire-and-forget logging
  postCreateNotify  (post-async, priority 1) ← fire-and-forget alert
```

When a hook is added, the generator:

1. Includes the hook stub in the generated `permissions.go` between `#HOOKS_START` and `#HOOKS_END` markers.
2. Includes the hook call in the `HandleCreate` method at the correct phase (pre-hooks before DAL forward, post-hooks after).
3. Sorts hooks by priority within each phase before generating the dispatch code.



Priority and async interaction: `preCreateValidate` runs first (priority 1, sync). It can return an error to abort the operation before the DAL is called. `preCreateAudit` runs second (priority 2, async) — it has already been dispatched to the workerpool by the time the main goroutine continues. If `preCreateValidate` fails on a later request, `preCreateAudit` will have already fired for that request. **Place sync validators at lower priority numbers than async hooks** to ensure validation runs before async work begins.

The conditional import logic is worth noting here. Because `preCreateAudit` and `postCreateNotify` are async, the TextGen template detects that the entity `needWorkerPool()` and includes the workerpool import in the generated file. If all hooks were sync, the workerpool import would be omitted entirely.

Step 3: Define Relations

```
forServer: motadata_user_management
entity: Roles

fields:
- ID      type: uuid   anotations: @ primaryKey
- desc    type: text   anotations: @ nullable
- name    type: string anotations: @ required

allowed_to:
- create
- update
- list
- delete
- get

preHooks:
  create
    - name: NotifyAdmin - priority: 10 isAsync: true
postHooks:
  << ... >>

relations:
relation Permission_pivot {
  with: Permissions

  operations:
    -assign
    -list
    -remove

  extra fields:
    << ... >>
}
```

A relation connecting Roles to Permissions is a separate concept in MPS, defined as a child of the Roles entity:

```
Entity: Roles
...
Relation: RolesPermissions
  from: Roles
  to:   Permissions
  operations: assign, remove, list
  extraFields:
    granted_at time auto
```

Compare to UserRoles:

```
Entity: User
...
Relation: UserRoles
  from: User
  to:   Roles
  operations: assign, remove, list
  extraFields:
    assigned_at  time  auto
```

The template is identical — only the entity names and field names differ. `RolesPermissions` generates into `roles.go` (because it is a child of `Roles`). `UserRoles` generates into `user.go` (because it is a child of `User`).

This gives three handler methods inside `roles.go`:

```
func (s *RolesHandler) HandleAssign(msg micro.Request) { ... }
func (s *RolesHandler) HandleRemove(msg micro.Request) { ... }
func (s *RolesHandler) HandleList(msg micro.Request)  { ... }
```

GO

With NATS subjects:

- `motadata.roles.permissions.assign`
- `motadata.roles.permissions.remove`
- `motadata.roles.permissions.list`

Step 4: Run TextGen

In MPS, right-click the root node → "Preview Generated Text" to see what will be produced. This is the fastest feedback loop — it shows the output inline without file I/O, without running `sync.sh`, without rebuilding Docker.

Use "Preview Generated Text" to verify:

- The struct fields match the declaration (correct types, correct annotations)
- The operation methods are present (`HandleCreate`, `HandleGet`, etc.)
- The hook calls appear in the correct phase order
- The `#HOOKS_START` / `#HOOKS_END` markers are present with the correct stub signatures

When the preview looks correct, trigger full TextGen (right-click → "Generate Text") to write the files to `source_gen/`.

MPS generates: * `source_gen/permissions.go` — Permissions entity handler (~307 lines) * Updated `source_gen/roles.go` — now includes RolesPermissions relation handlers * Updated `source_gen/sqlPrem_init_sql.sql` — now includes `iam.permissions` table (if `SqlSchem` references the new entity)



The SqlSchem reference gap: The SQL schema only generates tables for entities that are referenced in the `SqlSchem` root concept's `entityrefs` list. Adding `Permissions` as an entity does not automatically add it to the SQL schema. A reference to `Permissions` in the `SqlSchem` root must also be added. This was the cause of the missing `Permissions` table in the initial deployment — the entity existed, the handler worked, but the table was never created.

Step 5: Run sync.sh

`sync.sh` is the transition from "files on disk" to "running service." It is also the step with the most ways to go wrong.

```
./sync.sh
```

BASH

What happens, in order:

1. `permissions.go` is copied from `source_gen/` to `src/`. Previous `src/permissions.go` (if any) is overwritten.
2. `merge_hooks.py` runs. It finds `#HOOKS_START` in `src/permissions.go` and extracts three new stubs: `preCreateValidate`, `preCreateAudit`, `postCreateNotify`. These are appended to `userDefinedHooks.go`.
3. `roles.go` is also copied and merged — the `RolesPermissions` hooks appear as new stubs.
4. SDK is rsynced and patched.
5. Docker container is rebuilt and restarted.

After `sync.sh`, open `userDefinedHooks.go`:

```
// New stubs appended by merge_hooks.py:

func (s *PermissionsHandler) preCreateValidate(
    ctx context.Context, span tracer.Span, event *PermissionsCreatedEvent) error {
    return nil
}

func (s *PermissionsHandler) preCreateAudit(
    ctx context.Context, span tracer.Span, event *PermissionsCreatedEvent) {
}

func (s *PermissionsHandler) postCreateNotify(
    ctx context.Context, span tracer.Span, event *PermissionsCreatedEvent, data []byte) {
}
```

GO

The stubs are empty. Implement the business logic here. On the next `sync.sh` run, `merge_hooks.py` will skip these stubs (they already exist with matching signatures) and preserve the implementations.

War Story: The Trailing Dot Bug

During early development of the Permissions entity, a generated handler contained:

```
if event.Permissions. == "" {
    // validation
}
```

GO

A trailing dot — `event.Permissions.` — with nothing after it. The Go compiler caught this immediately, but the root cause was not obvious.

Tracing backward: the TextGen template iterates `node.fields` and for each non-auto, non-hidden, non-nullable field, appends:

```
append {|| event.} ;
append ${field.name.capitalize()} ;
append { == ""} ;
```

`field.name.capitalize()` returned an empty string. The field in MPS had been created in the editor but not named. The MPS editor allows a field to exist with an empty name — there is no constraint preventing it.

TextGen generated `event. + "" + `== ""` = event. == ""`.

The fix: give the field a name, or delete it.

Principle: Silent generation failures produce syntactically invalid code. The symptom appears in the compiler, not in MPS. When a Go compile error looks like a template expansion artifact (trailing dots, empty identifiers, doubled strings), trace backward to the TextGen node that produced that line — not to the Go code itself.

Step 6: Run demo.sh

```
./demo.sh
```

BASH

`demo.sh` connects to `nats://localhost:4229` and runs 21 tests. For the Permissions entity, it spawns a mock DAL responder:

```
nats reply "motadata.permissions.db.>" '{"status":"ok"}' &
```

BASH

Then sends test messages:

```
# Test: valid permission create
nats req motadata.permissions.create \
  '{"name":"read:users","description":"Can read user list"}'

# Test: missing required field (name)
nats req motadata.permissions.create \
  '{"description":"No name provided"}'

# Expected: validation error response
```

BASH

A passing test suite confirms:

- The NATS subjects are correctly registered (service discovery works)
- Validation fires on missing required fields
- Pre-hooks run in priority order
- DAL forwarding works (mock responder replies, main handler returns the reply)
- Post-hooks fire

What Can Go Wrong at Each Step

Step	Symptom	Root Cause
Define entity	Wrong field type in generated struct	Field declared with wrong DSL type; generator trusts the declaration

Step	Symptom	Root Cause
Define entity	Missing SQL table	Entity not referenced in <code>SqlSchem</code> root's <code>entityrefs</code> list
Define hooks	Async hook fires even when sync validator aborts	Async hook has lower priority number than sync validator — runs first
Run TextGen	Trailing dot in generated code	Field in MPS editor has empty name string
Run TextGen	<code>user_roless</code> double suffix in SQL	Hardcoded "s" suffix appended to entity name that already ends in 's'
Run sync.sh	Old hook implementation broken after sync	Hook changed from sync to async (signature mismatch); check <code>merge_hooks.py</code> stdout for warning
Run sync.sh	Build fails: cannot find module	SDK rsync didn't complete or module path patch failed; check <code>motadata-go-sdk/go.mod</code>
Run demo.sh	Test times out instead of failing	No mock DAL responder for the new entity's <code>.db.</code> subjects; add <code>nats reply</code> for the entity

Watch for:



- Running `demo.sh` before `sync.sh` completes. Docker Compose takes time to become healthy — `demo.sh` connects immediately and may see the old container.
- Implementing hook logic that depends on DAL response format before verifying the mock DAL response matches production DAL format. Mock responders return `{"status": "ok"}` for everything — production DAL returns entity-specific JSON.
- Forgetting that hook implementations in `userDefinedHooks.go` are not regenerated. If a hook is renamed in MPS, the old implementation remains in `userDefinedHooks.go` under the old name, unused. The new stub is appended with an empty body. The old implementation must be manually migrated and deleted.

The User Flow

```
preHooks:
  create
    - name: D - priority: 4 isAsync: false
    - name: C - priority: 3 isAsync: true
    - name: B - priority: 2 isAsync: false
    - name: M - priority: 9 isAsync: true
    - name: A - priority: 1 isAsync: true
  delete
    - name: Xyz - priority: 5 isAsync: false
    - name: Abc - priority: 4 isAsync: false
    - name: validateTen - priority: 3 isAsync: false
  update
    - name: notifyCache - priority: 2 isAsync: true
    - name: auditLog - priority: 1 isAsync: true
  get
    - name: traceAcces - priority: 1 isAsync: true
postHooks:
  update
    - name: invalidateCache - priority: 1 isAsync: true
  get
    - name: filterPII - priority: 3 isAsync: false
    - name: cacheResult - priority: 2 isAsync: true
    - name: enrichData - priority: 1 isAsync: false
  create
    - name: notifyAdmin - priority: 2 isAsync: true
    - name: sendWelcome - priority: 1 isAsync: true
  delete
    - name: mustDelete - priority: 2 isAsync: false

relations:
relation UserRoll {
  with: Roles

  operations:
    -assign
    -list
    -remove

  extra fields:
    - assignedAt type: time anotations: @ auto
}
```

A request enters the system as a NATS message. It exits as a NATS reply. Everything in between — unmarshaling, pre-hooks, validation, DAL forwarding, post-hooks — happens inside the UserManagement service in a single synchronous chain per handler method.

This section traces that chain from the caller's perspective.

The Subject Hierarchy

```
forServer: motadata_user_management
entity: Roles

fields:
- ID      type: uuid   anotations: @ primaryKey
- desc    type: text   anotations: @ nullable
- name    type: string anotations: @ required

allowed_to:
- create
- update
- list
- delete
- get

preHooks:
  create
    - name: NotifyAdmin - priority: 10 isAsync: true
postHooks:
  << ... >>

relations:
relation Permission_pivot {
  with: Permissions

  operations:
    -assign
    -list
    -remove

  extra fields:
    << ... >>
}
```

The service registers endpoints on a tree of NATS subjects. Every subject follows the pattern `motadata.{entity}.{operation}`.

The full set of registered subjects for the three entities and two relations:

```

# Entity operations (15 total)
motadata.user.create
motadata.user.get
motadata.user.update
motadata.user.delete
motadata.user.list

motadata.roles.create
motadata.roles.get
motadata.roles.update
motadata.roles.delete
motadata.roles.list

motadata.permissions.create
motadata.permissions.get
motadata.permissions.update
motadata.permissions.delete
motadata.permissions.list

# Relation operations (6 total)
motadata.user.roles.assign
motadata.user.roles.remove
motadata.user.roles.list

motadata.roles.permissions.assign
motadata.roles.permissions.remove
motadata.roles.permissions.list

```

Twenty-one registered endpoints. Each maps to one handler method in the generated Go code.

The DAL subjects follow the same pattern with `.db.` inserted:

```

motadata.user.create      → motadata.user.db.create
motadata.roles.list       → motadata.roles.db.list
motadata.user.roles.assign → motadata.user.roles.db.assign

```

The service is transparent in both directions: callers use the business subjects, the DAL uses the `db` subjects, and the service bridges them.

Tracing One Request End-to-End

Scenario: a caller creates a new user.

```

nats req motadata.user.create \
  '{"username":"alice","user_mail":"alice@example.com","password":"s3cr3t"}'

```

BASH

Step 1: NATS delivers the message. The NATS micro service routes the message to `UserHandler.HandleCreate` based on the registered subject.

Step 2: OTEL span starts.

```

ctx, span := s.tracer.Start(ctx, "user.create")
defer span.End()

```

GO

Everything from this point happens within the span.

Step 3: Unmarshal.

```

var event UserCreatedEvent
if err := json.Unmarshal(msg.Data(), &event); err != nil {
    msg.Error("400", "invalid JSON", nil)
    return
}

```

If the body is not valid JSON, the handler replies immediately with a 400 error. No hooks run.

Step 4: Pre-hooks run in priority order.

```

// Pre-sync hook (priority 1) – can abort
if err := s.preCreateD(ctx, span, &event); err != nil {
    msg.Error("422", err.Error(), nil)
    return
}

// Pre-async hook (priority 2) – fire-and-forget
_ = workerpool.AsyncWithCtx(ctx, s.wp, func() {
    s.preCreateM(ctx, span, &event)
})

```

The sync hook runs first. If it returns an error, the handler aborts — the async hook has not been dispatched yet and does not run. The async hook fires after the sync hook passes, dispatched to the workerpool; the main goroutine continues immediately.

Step 5: Validate required fields.

```

if event.UserName == "" || event.UserMail == "" {
    msg.Error("400", "missing required fields", nil)
    return
}

```

Generated validation — TextGen iterates non-auto, non-hidden, non-nullable fields and builds this condition. Password is excluded (it is `hidden`). ID is excluded (it is `primaryKey` and `auto`).

Step 6: Set span attributes.

```

span.SetAttributes(
    attribute.String("user.username", event.UserName),
    attribute.String("user.email", event.UserMail),
)

```

Step 7: Forward to DAL.

```

dalResp, err := s.publisher.Request(
    ctx,
    "motadata.user.db.create",
    requestBody,
    headers,
    30*time.Second,
)
if err != nil {
    msg.Error("503", "DAL timeout", nil)
    return
}

```

Synchronous NATS request-reply with a 30-second timeout. The handler blocks until the DAL responds or the timeout fires.

Step 8: Post-hooks run.

```
// Post-sync hook – transforms response
responseData = s.postGetFilterPII(ctx, span, &event, dalResp.Data())

// Post-async hook – fire-and-forget
_ = workerpool.AsyncWithCtx(ctx, s.wp, func() {
    s.postCreateNotifyAdmin(ctx, span, &event, responseData)
})
```

Post-sync hooks receive the DAL response as []byte and return the (possibly modified) []byte. Post-async hooks receive the response data but their return value is discarded.

Step 9: Reply to caller.

```
msg.Respond(responseData)
```

The caller receives the DAL’s response, possibly transformed by post-sync hooks.

Hook Execution Model

	Pre-operation	Post-operation
Sync	Runs in-line. Returns error to abort. Caller waits.	Runs in-line. Receives []byte, returns modified []byte. Caller waits.
Async	Dispatched to workerpool. Returns nothing. Caller does not wait.	Dispatched to workerpool. Receives []byte read-only. Caller does not wait.

Practical implications:

- **Validation belongs in pre-sync.** Only pre-sync hooks can prevent the operation from reaching the DAL.
- **Audit logging belongs in pre-async or post-async.** It should not block the operation.
- **PII filtering belongs in post-sync.** It transforms the response before the caller sees it.
- **Cache invalidation belongs in post-async.** It runs after the caller receives the response.



Async hook error handling: The line `_ = workerpool.AsyncWithCtx(...)` discards the workerpool’s return value. Errors inside async hooks are swallowed — they do not propagate to the caller and do not abort the operation. This is intentional: fire-and-forget means the caller’s flow is decoupled from the hook’s outcome. If an async hook must report errors, it must do so internally (log, metric, alert) — not by returning an error.

Service Discovery

The NATS micro protocol provides built-in service discovery:

```
nats req '$SRV.INFO' ''
```

BASH

The response includes the service name, version, and the full list of registered endpoints. Test #21 in `demo.sh` verifies this — service discoverability is the final check after all 20 entity and relation tests pass.

The service name in the response derives from the DSL's `NatsServer` concept → first-line comment in `main.go` → `sync.sh` extraction → Docker build argument. One declaration in MPS populates the discovery response.

Error Patterns

Three categories of errors:

Error	Caller Response	Root Cause
Invalid JSON	400 invalid JSON immediately	<code>json.Unmarshal</code> fails; no hooks run
Validation failure	400 missing required fields or hook-specific error	Required field empty, or pre-sync hook returns error
DAL timeout	503 DAL timeout after 30 seconds	DAL did not respond within the timeout window

There is no retry logic. A DAL timeout means the caller receives a 503 and must decide whether to retry. The service does not retry on the caller's behalf.



Watch for:

- Pre-async hooks that fire before pre-sync validation. If the async hook starts an irreversible action (sending an email, writing an audit record) and the sync validator then aborts the operation, the irreversible action has already happened. Priority ordering is the only protection — place sync validators at lower priority numbers (runs first).
- Post-sync hooks that panic. A panic in a post-sync hook crashes the request goroutine; the caller never receives a response and the connection times out rather than getting a clean error.
- Long-running pre-sync hooks. Because they block the request path, a slow pre-sync hook directly increases response latency for every caller. Use pre-async for anything that might take more than a few milliseconds.

Lessons and Red Flags

Every system reveals failure modes after it is built. This section documents the ones this system revealed — as specific incidents with forensics and named signals.

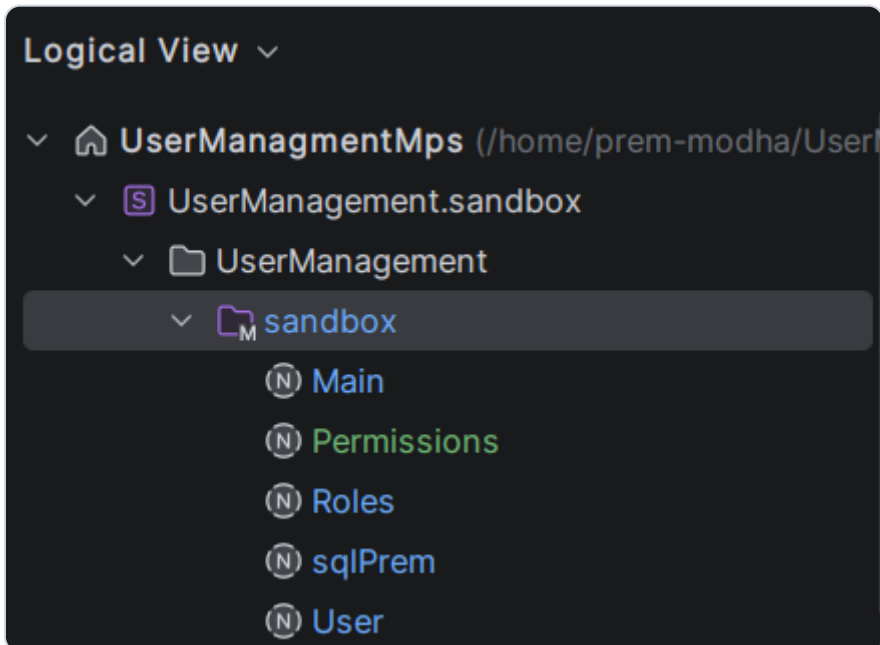
Each follows the same four-part structure:

1. **Incident** — what went wrong, exactly

2. **Forensics** — how the root cause was found
3. **Principle** — the generalizable rule
4. **Signal** — how to recognize the pattern before it breaks

Lesson 1: The Single-File Trap

Incident

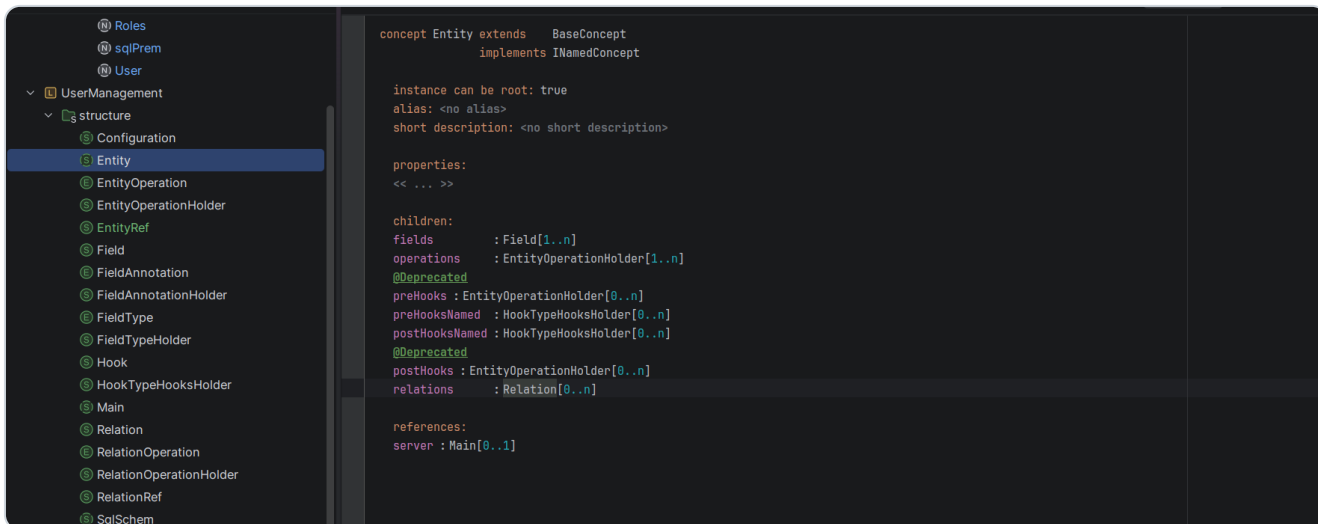


UMAN generated working Go code. The code compiled, the tests passed (such as they were), and the Prisma-style schema definition felt natural. But all entities and their handlers were in a single generated file.

When a fourth entity was added, the single file grew past 1,000 lines. When a template bug was found, fixing it required mentally tracking which section of the 1,000-line file corresponded to which entity.

The project was abandoned — not because generation failed, but because the output structure was unmaintainable.

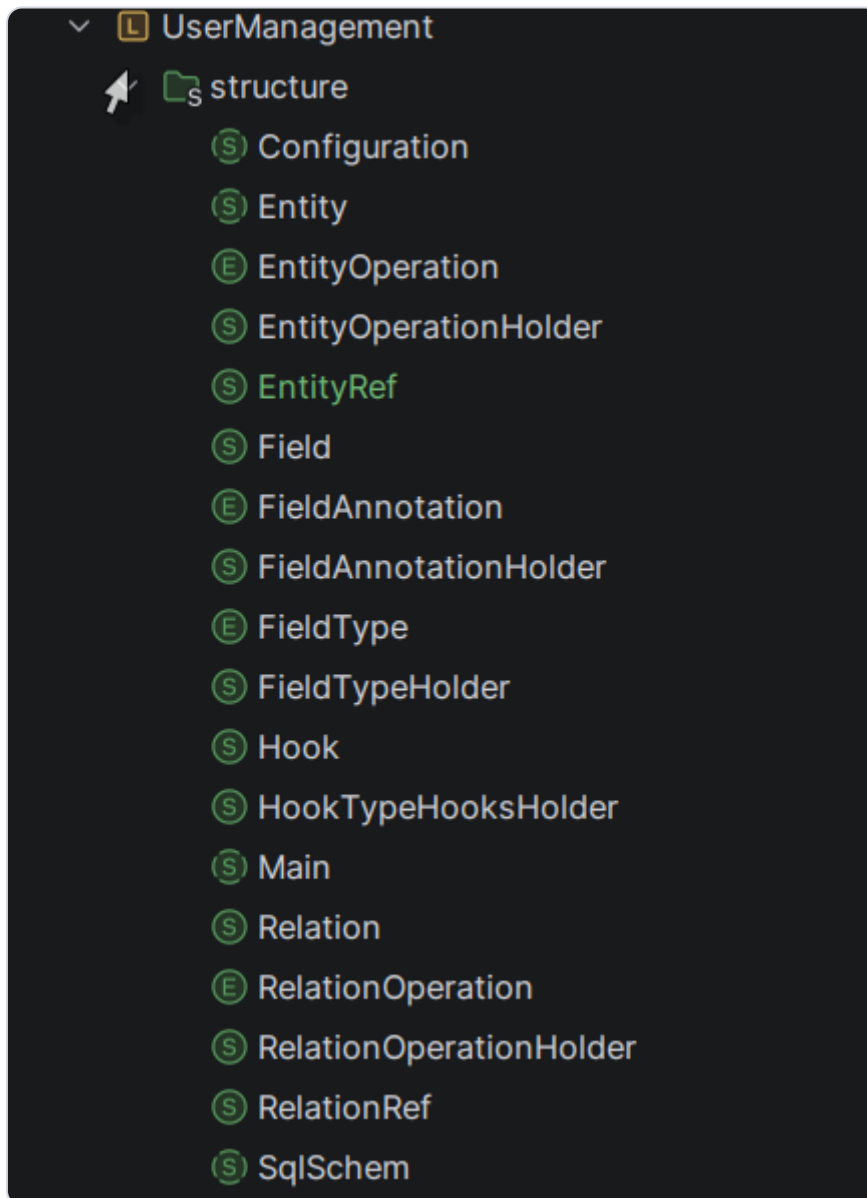
Forensics



The root cause was the MPS one-concept-one-file constraint, but the trigger was choosing a Prisma-style single-model definition. Prisma works because it generates a single schema file and all code is human-readable and separate. MPS TextGen puts all output from a single root concept into one file — the organizational discipline of Prisma (one schema, many generated files) does not translate.

The same mistake appeared in `pcrd` : entity/relation declarations, Go Gin server, SQL schema, Docker setup — all generated and working, all in one file. Two working systems, both abandoned for the same structural reason.

Principle



The generated output structure mirrors the DSL model structure.

If the DSL model puts everything in one root concept, the generated output is one file. To get N files, N root concepts are required — one per generated output.

This is not a limitation to work around. It is the MPS architecture. Design the DSL model to produce the output structure you want, not the structure that feels natural to declare.

Signal

The trap is active when:

- The generated file grows proportionally to entity count
- Searching within a generated file for "the Roles section" becomes necessary
- Adding a new entity requires scrolling to the right place in the generated output

The fix: split the DSL model into separate root concepts, one per desired output file.

Lesson 2: The Hook Name Collision

```
preHooks:
  create
    - name: D - priority: 4 isAsync: false
    - name: C - priority: 3 isAsync: true
    - name: B - priority: 2 isAsync: false
    - name: M - priority: 9 isAsync: true
    - name: A - priority: 1 isAsync: true
  delete
    - name: Xyz - priority: 5 isAsync: false
    - name: Abc - priority: 4 isAsync: false
    - name: validateTen - priority: 3 isAsync: false
  update
    - name: notifyCache - priority: 2 isAsync: true
    - name: auditLog - priority: 1 isAsync: true
  get
    - name: traceAcces - priority: 1 isAsync: true
postHooks:
  update
    - name: invalidateCache - priority: 1 isAsync: true
  get
    - name: filterPII - priority: 3 isAsync: false
    - name: cacheResult - priority: 2 isAsync: true
    - name: enrichData - priority: 1 isAsync: false
  create
    - name: notifyAdmin - priority: 2 isAsync: true
    - name: sendWelcome - priority: 1 isAsync: true
  delete
    - name: mustDelete - priority: 2 isAsync: false

relations:
relation UserRoll {
  with: Roles

  operations:
    -assign
    -list
    -remove

  extra fields:
    - assignedAt type: time anotations: @ auto
}
```

Incident

```
forServer: motadata_user_management
entity: Permissions

fields:
- ID      type: uuid  anotations: @ required
                                     @ primaryKey
                                     @ immutable
- name    type: string anotations: @ required
- module  type: text   anotations: @ required
- can     type: string anotations: @ required

allowed_to:
- create
- update
- delete
- list
- get

preHooks:
  create
    - name: NotifyManager - priority: 100  isAsync: false
    - name: NotifyAdmin   - priority: 1000 isAsync: true
postHooks:
  create
    - name: audit - priority: 1  isAsync: true

relations:
<< ... >>
```

`merge_hooks.py` keyed hook stubs by function name. When `UserRoles` and `RolesPermissions` both had a hook named `preAssignHook`, the second hook's stub overwrote the first in `userDefinedHooks.go`.

The `UserRoles` implementation was silently replaced with the `RolesPermissions` stub. The `UserRoles` assign operation then did nothing — the hook ran but used the empty stub.

Forensics

The merge script compared function names: `preAssignHook == preAssignHook`. Both hooks had the same name. The merge logic found a "match" and skipped the second stub, which meant the first entity's implementation was treated as the correct implementation for both.

The receiver type was not part of the identity key: `*UserHandler.preAssignHook` and `*RolesHandler.preAssignHook` are different functions, but the script saw them as `preAssignHook` and `preAssignHook` — identical.

Principle

Identity requires context.

A function name is not a unique identifier in a codebase with multiple receiver types. `preAssignHook` on `*UserHandler` and `preAssignHook` on `*RolesHandler` are different functions that happen to share a name. The full identity is `ReceiverType.FunctionName`.

The fix: key stubs by `*ReceiverType.funcName` (e.g., `*UserHandler.preAssignHook` vs. `*RolesHandler.preAssignHook`).

Signal

The trap is active when:

- Merge or matching logic uses function names without qualifying the containing type or scope
- Two different entities have hooks with the same name and different implementations
- Silently correct-looking output (no error, just wrong behavior) appears after a merge

Lesson 3: Presence Is Not Equivalence

Incident

A hook was changed from post-sync to post-async. The return type changed from `[]byte` to nothing. The name stayed the same.

`merge_hooks.py` found the hook in `userDefinedHooks.go`, saw the name matched, and kept the existing implementation — with the old return type.

The Go compiler reported a type error. The error message pointed to the hook implementation in `userDefinedHooks.go`, not to the merge script that caused the problem.

Forensics

The merge script checked: "does this function name already exist in `userDefinedHooks.go`?" It did. The script concluded: "this is an existing hook, preserve it."

But "exists" is not the same as "correct shape." The existing function had the wrong signature for the new hook type. The merge script had no concept of signature validation — it only checked for name presence.

Principle

Presence is not equivalence.

Finding a function by name does not mean the function is the right function. A hook with the correct name and the wrong signature is not a match — it is a collision waiting to break the build.

The fix: compare signatures on every merge, not just names. When a signature mismatch is detected, print the old implementation as a warning and replace the stub.

Signal

The trap is active when:

- Existence checks are the only form of identity validation
 - A script that merges or updates code checks "does it exist?" but not "does it look right?"
 - Build errors appear in preserved user code, not in generated code
-

Lesson 4: Silent Generation Failures

Incident

A generated handler contained:

```
if event.Permissions. == "" {
```

GO

A trailing dot — `event.Permissions.` — with nothing after it. The Go compiler reported a syntax error. The error pointed to the generated file.

The generated file is not supposed to be edited. When a generated file has a bug, the fix must be in the generator — not in the file itself.

Forensics

TextGen produces this line from:

```
append {|| event.} ;  
append ${field.name.capitalize()} ;  
append { == ""} ;
```

`field.name.capitalize()` returned an empty string. The field in MPS had been created in the editor but not named. The MPS editor allows a field to exist with an empty name — there is no constraint preventing it.

TextGen generated `event. + "" + `== ``` = event. == ""`.

Principle

Silent generation failures produce syntactically invalid output.

When TextGen expands a template with an empty or missing value, it produces whatever character combination results from the concatenation. There is no TextGen equivalent of a null check or a "required field" error during generation.

Two protections are available:

1. Add MPS constraints to the concept (e.g., "name must be non-empty") — the editor will show an error before generation runs.
2. Add `System.out.println(field.name)` inside the TextGen template to log intermediate values to the MPS Messages console during generation.

Signal

The trap is active when:

- A generated Go file has a syntax error that looks like a template artifact (trailing dots, empty identifiers, doubled strings)
- The error points to a line that no human wrote
- The instinct is to edit the generated file to fix it

The fix is always in the generator — either the DSL model has a missing value, or the TextGen template is not handling empty values defensively.

Lesson 5: Don't Fight the Walled Garden — Build a Bridge

Incident

Writing the 300- `append`-statement Entity TextGen template character by character in MPS's projectional editor took weeks. Every string interpolation required navigating completion menus. Copy-paste from an external editor was impossible. AI assistance was unavailable. When a bug appeared on line 47 of the generated output, there was no way to set a breakpoint or trace which `append` statement produced that line.

The first escape attempt — switching to the DSL Foundry TextGen plugin — failed for lack of documentation and incomprehensible groupings.

Forensics

The root cause was accepting the walled garden as a fixed constraint. "This must be typed in MPS" became the unquestioned premise.

The correct question was: "what is MPS actually doing when TextGen runs?" The answer: MPS reads `.mps` XML files and interprets the TextGen node structure they contain. The `.mps` format is the real input — the projectional editor is just one way to produce it.

Principle

When a tool's editing UX is the blocker, build a bridge to it rather than fighting within it.

The walled garden walls are high on the inside. From the outside, the `.mps` file format is just XML. A Python script can write valid XML. The bridge (`parse_textgen.py`) took one day to build and eliminated weeks of manual authoring for every future template.

Signal

The trap is active when:

- Multiple days have been spent manually authoring something that could be expressed in a text file
- The tool does not allow import from external formats
- Plugin switching has been tried and failed

The pattern name: **reverse-engineer the import format**. Find out what the tool actually reads, not what it displays. Build a converter that produces that format from something editable externally.

Lesson 6: External Scripts Are First-Class Tooling

```
forServer: motadata_user_management
entity: User
```

fields:

```
- ID          type: uuid      anotations: @ primaryKey
- password    type: password  anotations: @ required
                                     @ hidden
- UserName    type: string    anotations: @ required
- userMail    type: email     anotations: @ unique
- createdAt   type: time      anotations: @ auto
```

allowed_to:

```
- create
- update
- delete
- get
- list
```

preHooks:

create

```
- name: D - priority: 4 isAsync: false
- name: C - priority: 3 isAsync: true
- name: B - priority: 2 isAsync: false
- name: M - priority: 9 isAsync: true
- name: A - priority: 1 isAsync: true
```

delete

```
- name: Xyz          - priority: 5 isAsync: false
- name: Abc          - priority: 4 isAsync: false
- name: validateTen - priority: 3 isAsync: false
```

update

```
- name: notifyCache - priority: 2 isAsync: true
- name: auditLog    - priority: 1 isAsync: true
```

get

```
- name: traceAcces - priority: 1 isAsync: true
```

postHooks:

```
..
```

agmentMps > UserManagement.sandbox > sandbox > User > ⓘ Field

Incident

`userDefinedHooks.go` needed to exist as a separate file from generated entity handlers. The MPS one-concept-one-file constraint said: to generate a separate file, create a separate root concept. Creating a root concept just for hook stubs would add a trivial one-liner to the sandbox — cluttering it with noise.

The same problem applied to the SQL schema file: the `SqlSchem` root concept needs explicit `entityrefs` to include each entity. Forgetting to add a reference means the table is not generated — silently.

Forensics

Both problems share a structure: MPS’s generation model produces files but cannot *process* or *coordinate* the output of multiple generation passes. It cannot extract hooks from all entity files and combine them into one file. It cannot verify that all entities have been referenced in `SqlSchem`.

External scripts have no such limitation.

Principle

External scripts are first-class tooling, not workarounds.

`merge_hooks.py` is not a hack around MPS’s limitations. It is the right tool for the job MPS cannot do: extract, merge, and validate across multiple generated files. `sync.sh` is not a workaround for the SDK registry gap — it is the right tool for coordinating the full pipeline.

The pattern: generate inside MPS → transform and coordinate outside MPS.

Signal

The trap is active when:

- A new MPS root concept is being created just to generate a file that combines or processes output from other concepts
- A build failure happens because two separately generated files are inconsistent
- Post-processing logic is being added inside a TextGen template

Principles Reference Card

Name	Principle	Watch For
The Single-File Trap	Generated output structure mirrors DSL model structure	Generated file growing with entity count; searching within generated output
Identity Requires Context	<code>ReceiverType.FunctionName</code> is the minimum unique identity for a hook	Name-only matching in merge scripts; silent behavior changes after merge
Presence Is Not Equivalence	Name match without signature match is a collision, not a hit	Existence-only checks; build errors in preserved user code
Silent Generation Failures	TextGen produces whatever concatenation results from template + values	Trailing dots, empty identifiers, doubled strings in generated code

Name	Principle	Watch For
Bridge the Walled Garden	Reverse-engineer the tool's import format; build a converter	Weeks of manual authoring; failed plugin switches; no AI assistance
External Scripts Are First-Class	Generate inside MPS, coordinate outside MPS	New root concepts for post-processing; inconsistency between generated files



Final watch-for:

Scoping in MPS remains partially understood. The current DSL sidesteps complex scoping by using entity references by name rather than variable binding. If future DSL features require computed fields, conditional logic in templates, or cross-entity references at generation time, scoping will need to be properly mastered. The `ScopeProvider.getScope()` mechanism exists but was not needed for this project — and "did not need it" is not the same as "understood it." Do not assume the scoping knowledge from learning projects (`gtog`, `calculator`) is sufficient for production use without verification.